AI USAGE REPORT

Model used: ChatGPT
Most of the project was assisted by AI and I only made some small changes. Design ideas are mine.

Prompt: “Need template for Chatroom using Firebase”
Given Template:

Src/firebase.js:

```
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getFirestore } from "firebase/firestore";

const firebaseConfig = {
  apiKey: "PASTE_YOUR_API_KEY_HERE",
  authDomain: "PASTE_YOUR_AUTH_DOMAIN_HERE",
  projectId: "PASTE_YOUR_PROJECT_ID_HERE",
  storageBucket: "PASTE_YOUR_STORAGE_BUCKET_HERE",
  messagingSenderId: "PASTE_YOUR_SENDER_ID_HERE",
  appId: "PASTE_YOUR_APP_ID_HERE"
};

const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);
export const db = getFirestore(app);
```

src/App.js:

```
import React, { useEffect, useState, useRef } from "react";
import "./App.css";

import { auth, db } from "./firebase";

import {
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signOut,
  onAuthStateChanged
} from "firebase/auth";

import {
  collection,
  doc,
  setDoc,
  addDoc,
  getDocs,
  getDoc,
  updateDoc,
  deleteDoc,
  query,
  where,
  orderBy,
  onSnapshot,
  serverTimestamp
} from "firebase/firestore";

function App() {
  const [user, setUser] = useState(null);
  const [mode, setMode] = useState("login");
```

```
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");

const [profile, setProfile] = useState(null);
const [showProfile, setShowProfile] = useState(false);

const [chatrooms, setChatrooms] = useState([]);
const [currentRoom, setCurrentRoom] = useState(null);
const [messages, setMessages] = useState([]);

const [newRoomName, setNewRoomName] = useState("");
const [inviteEmail, setInviteEmail] = useState("");
const [messageText, setMessageText] = useState("");
const [searchText, setSearchText] = useState("");

const [editingMessage, setEditingMessage] = useState(null);
const [editingText, setEditingText] = useState("");

const bottomRef = useRef(null);

useEffect(() => {
  const unsubscribe = onAuthStateChanged(auth, async currentUser => {
    setUser(currentUser);

    if (currentUser) {
      await loadProfile(currentUser.uid);
    } else {
      setProfile(null);
    }
  });

  return () => unsubscribe();
}, []);

useEffect(() => {
  if (!user) return;

  const q = query(
    collection(db, "chatrooms"),
    where("members", "array-contains", user.uid)
  );

  const unsubscribe = onSnapshot(q, snapshot => {
    const rooms = snapshot.docs.map(docSnap => ({
      id: docSnap.id,
      ...docSnap.data()
    }));

    setChatrooms(rooms);
  });

  return () => unsubscribe();
}, [user]);

useEffect(() => {
  if (!currentRoom) return;

  const q = query(
    collection(db, "chatrooms", currentRoom.id, "messages"),
    orderBy("createdAt", "asc")
  );

  const unsubscribe = onSnapshot(q, snapshot => {
    const msgList = snapshot.docs.map(docSnap => ({
      id: docSnap.id,
      ...docSnap.data()
    }));

    setMessages(msgList);
  });
```

```
  return () => unsubscribe();
}, [currentRoom]);

useEffect(() => {
  bottomRef.current?.scrollIntoView({ behavior: "smooth" });
}, [messages]);

async function loadProfile(uid) {
  const profileRef = doc(db, "users", uid);
  const profileSnap = await getDoc(profileRef);

  if (profileSnap.exists()) {
    setProfile(profileSnap.data());
  }
}

async function handleSignup(e) {
  e.preventDefault();

  const result = await createUserWithEmailAndPassword(auth, email, password);

  const newProfile = {
    uid: result.user.uid,
    email: result.user.email,
    username: result.user.email.split("@")[0],
    phone: "",
    address: "",
    profilePicture: ""
  };

  await setDoc(doc(db, "users", result.user.uid), newProfile);

  setProfile(newProfile);
  setEmail("");
  setPassword("");
}

async function handleLogin(e) {
  e.preventDefault();

  await signInWithEmailAndPassword(auth, email, password);

  setEmail("");
  setPassword("");
}

async function handleLogout() {
  await signOut(auth);
  setCurrentRoom(null);
}

async function saveProfile(updatedProfile) {
  await updateDoc(doc(db, "users", user.uid), updatedProfile);
  setProfile(updatedProfile);
  setShowProfile(false);
}

async function createRoom() {
  if (!newRoomName.trim()) return;

  await addDoc(collection(db, "chatrooms"), {
    name: newRoomName,
    members: [user.uid],
    memberEmails: [user.email],
    createdBy: user.uid,
    createdAt: serverTimestamp()
  });

  setNewRoomName("");
```

```
}

async function inviteMember() {
  if (!currentRoom || !inviteEmail.trim()) return;

  const q = query(
    collection(db, "users"),
    where("email", "==", inviteEmail.trim())
  );

  const snapshot = await getDocs(q);

  if (snapshot.empty) {
    alert("No registered user found with this email.");
    return;
  }

  const invitedUser = snapshot.docs[0].data();

  if (currentRoom.members.includes(invitedUser.uid)) {
    alert("This user is already in the chatroom.");
    return;
  }

  const updatedMembers = [...currentRoom.members, invitedUser.uid];
  const updatedEmails = [...currentRoom.memberEmails, invitedUser.email];

  await updateDoc(doc(db, "chatrooms", currentRoom.id), {
    members: updatedMembers,
    memberEmails: updatedEmails
  });

  setCurrentRoom({
    ...currentRoom,
    members: updatedMembers,
    memberEmails: updatedEmails
  });

  setInviteEmail("");
}

async function sendMessage() {
  if (!currentRoom || !messageText.trim()) return;

  await addDoc(collection(db, "chatrooms", currentRoom.id, "messages"), {
    text: messageText,
    senderId: user.uid,
    senderEmail: user.email,
    senderName: profile?.username || user.email,
    senderPhoto: profile?.profilePicture || "",
    type: "text",
    createdAt: serverTimestamp(),
    edited: false
  });

  setMessageText("");
}

async function sendImage(e) {
  const file = e.target.files[0];
  if (!file || !currentRoom) return;

  const reader = new FileReader();

  reader.onload = async event => {
    await addDoc(collection(db, "chatrooms", currentRoom.id, "messages"), {
      image: event.target.result,
      senderId: user.uid,
      senderEmail: user.email,
      senderName: profile?.username || user.email,
```

```
      senderPhoto: profile?.profilePicture || "",
      type: "image",
      createdAt: serverTimestamp(),
      edited: false
    });
  };

  reader.readAsDataURL(file);
  e.target.value = "";
}

async function unsendMessage(messageId, senderId) {
  if (senderId !== user.uid) {
    alert("You can only unsend your own message.");
    return;
  }

  await deleteDoc(doc(db, "chatrooms", currentRoom.id, "messages", messageId));
}

function startEdit(message) {
  if (message.senderId !== user.uid) {
    alert("You can only edit your own message.");
    return;
  }

  setEditingMessage(message);
  setEditingText(message.text || "");
}

async function saveEdit() {
  if (!editingMessage || !editingText.trim()) return;

  await updateDoc(
    doc(db, "chatrooms", currentRoom.id, "messages", editingMessage.id),
    {
      text: editingText,
      edited: true
    }
  );

  setEditingMessage(null);
  setEditingText("");
}

const filteredMessages = messages.filter(message => {
  if (!searchText.trim()) return true;

  return (
    message.text &&
    message.text.toLowerCase().includes(searchText.toLowerCase())
  );
});

if (!user) {
  return (
    <div className="auth-page">
      <div className="auth-card">
        <h1>Firebase Chatroom</h1>

        <div className="tab-row">
          <button
            className={mode === "login" ? "active" : ""}
            onClick={() => setMode("login")}
          >
            Sign In
          </button>

          <button
            className={mode === "signup" ? "active" : ""}
```

```
            onClick={() => setMode("signup")}
          >
            Sign Up
          </button>
        </div>

        <form onSubmit={mode === "login" ? handleLogin : handleSignup}>
          <input
            type="email"
            placeholder="Email"
            value={email}
            onChange={e => setEmail(e.target.value)}
            required
          />

          <input
            type="password"
            placeholder="Password"
            value={password}
            onChange={e => setPassword(e.target.value)}
            required
          />

          <button type="submit">
            {mode === "login" ? "Sign In" : "Create Account"}
          </button>
        </form>
      </div>
    </div>
  );
}

return (
  <div className="app">
    <aside className="sidebar">
      <div className="user-box">
        {profile?.profilePicture ? (
          <img src={profile.profilePicture} alt="profile" />
        ) : (
          <div className="avatar">{user.email[0].toUpperCase()}</div>
        )}

        <div>
          <strong>{profile?.username || user.email}</strong>
          <p>{user.email}</p>
        </div>
      </div>

      <button onClick={() => setShowProfile(true)}>Edit Profile</button>
      <button onClick={handleLogout}>Logout</button>

      <hr />

      <h2>Chatrooms</h2>

      <div className="create-room">
        <input
          placeholder="New room name"
          value={newRoomName}
          onChange={e => setNewRoomName(e.target.value)}
        />
        <button onClick={createRoom}>Create</button>
      </div>

      <div className="room-list">
        {chatrooms.map(room => (
          <button
            key={room.id}
            className={currentRoom?.id === room.id ? "room active-room" : "room"}
            onClick={() => setCurrentRoom(room)}
```

```
      >
        {room.name}
      </button>
    ))}
  </div>
</aside>

<main className="chat-area">
  {!currentRoom ? (
    <div className="empty-chat">
      <h2>Select or create a chatroom</h2>
    </div>
  ) : (
    <>
      <header className="chat-header">
        <div>
          <h2>{currentRoom.name}</h2>
          <p>{currentRoom.memberEmails?.join(", ")}</p>
        </div>

        <div className="invite-box">
          <input
            placeholder="Invite by email"
            value={inviteEmail}
            onChange={e => setInviteEmail(e.target.value)}
          />
          <button onClick={inviteMember}>Invite</button>
        </div>
      </header>

      <div className="search-box">
        <input
          placeholder="Search messages..."
          value={searchText}
          onChange={e => setSearchText(e.target.value)}
        />
      </div>

      <div className="message-list">
        {filteredMessages.map(message => {
          const isMine = message.senderId === user.uid;

          return (
            <div
              key={message.id}
              className={isMine ? "message-row mine" : "message-row"}
            >
              <div className="message-bubble">
                <div className="message-sender">
                  {message.senderPhoto && (
                    <img src={message.senderPhoto} alt="sender" />
                  )}
                  <span>{message.senderName || message.senderEmail}</span>
                </div>

                {message.type === "image" ? (
                  <img
                    className="sent-image"
                    src={message.image}
                    alt="sent"
                  />
                ) : (
                  <p>{message.text}</p>
                )}

                {message.edited && <small>edited</small>}

                {isMine && (
                  <div className="message-actions">
                    {message.type === "text" && (
```

```
                  <button onClick={() => startEdit(message)}>
                    Edit
                  </button>
                )}

                <button
                  onClick={() =>
                    unsendMessage(message.id, message.senderId)
                  }
                >
                  Unsend
                </button>
              </div>
            )}
          </div>
        </div>
      );
    })}

    <div ref={bottomRef}></div>
  </div>

  <footer className="chat-input">
    <input
      type="file"
      accept="image/*"
      onChange={sendImage}
    />

    <input
      placeholder="Type a message..."
      value={messageText}
      onChange={e => setMessageText(e.target.value)}
      onKeyDown={e => {
        if (e.key === "Enter") sendMessage();
      }}
    />

    <button onClick={sendMessage}>Send</button>
  </footer>
 </>
)}
</main>

{showProfile && (
 <ProfileModal
  profile={profile}
  onClose={() => setShowProfile(false)}
  onSave={saveProfile}
 />
)}

{editingMessage && (
 <div className="modal-bg">
  <div className="modal">
   <h2>Edit Message</h2>

   <textarea
    value={editingText}
    onChange={e => setEditingText(e.target.value)}
   />

   <div className="modal-buttons">
    <button onClick={saveEdit}>Save</button>
    <button onClick={() => setEditingMessage(null)}>Cancel</button>
   </div>
  </div>
 </div>
)}
</div>
```

```
  );
}

function ProfileModal({ profile, onClose, onSave }) {
  const [username, setUsername] = useState(profile?.username || "");
  const [email, setEmail] = useState(profile?.email || "");
  const [phone, setPhone] = useState(profile?.phone || "");
  const [address, setAddress] = useState(profile?.address || "");
  const [profilePicture, setProfilePicture] = useState(
    profile?.profilePicture || ""
  );

  function handleImage(e) {
    const file = e.target.files[0];
    if (!file) return;

    const reader = new FileReader();

    reader.onload = event => {
      setProfilePicture(event.target.result);
    };

    reader.readAsDataURL(file);
  }

  function handleSave() {
    onSave({
      ...profile,
      username,
      email,
      phone,
      address,
      profilePicture
    });
  }

  return (
    <div className="modal-bg">
      <div className="modal">
        <h2>User Profile</h2>

        {profilePicture && (
          <img className="profile-preview" src={profilePicture} alt="profile" />
        )}

        <input type="file" accept="image/*" onChange={handleImage} />

        <input
          placeholder="Username"
          value={username}
          onChange={e => setUsername(e.target.value)}
        />

        <input
          placeholder="Email"
          value={email}
          onChange={e => setEmail(e.target.value)}
        />

        <input
          placeholder="Phone number"
          value={phone}
          onChange={e => setPhone(e.target.value)}
        />

        <input
          placeholder="Address"
          value={address}
          onChange={e => setAddress(e.target.value)}
        />
```

```
        <div className="modal-buttons">
          <button onClick={handleSave}>Save</button>
          <button onClick={onClose}>Cancel</button>
        </div>
      </div>
    </div>
  );
}

export default App;
```

src/App.css:

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f3f4f6;
}

button {
  cursor: pointer;
}

.auth-page {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  background: linear-gradient(135deg, #dbeafe, #fce7f3);
}

.auth-card {
  width: 360px;
  background: white;
  padding: 24px;
  border-radius: 18px;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.12);
  animation: fadeIn 0.5s ease;
}

.auth-card h1 {
  text-align: center;
  font-size: 26px;
}

.auth-card input,
.auth-card button {
  width: 100%;
  padding: 12px;
  margin-top: 12px;
  border-radius: 10px;
  border: 1px solid #ddd;
}

.auth-card button {
  background: #2563eb;
  color: white;
  border: none;
}

.tab-row {
  display: flex;
  gap: 10px;
}

.tab-row button {
```

```
  background: #e5e7eb;
  color: #111827;
}

.tab-row button.active {
  background: #2563eb;
  color: white;
}

.app {
  height: 100vh;
  display: flex;
  overflow: hidden;
}

.sidebar {
  width: 300px;
  background: #111827;
  color: white;
  padding: 16px;
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.sidebar button {
  padding: 10px;
  border: none;
  border-radius: 10px;
  background: #374151;
  color: white;
}

.user-box {
  display: flex;
  align-items: center;
  gap: 12px;
}

.user-box img,
.avatar {
  width: 44px;
  height: 44px;
  border-radius: 50%;
}

.user-box img {
  object-fit: cover;
}

.avatar {
  background: #2563eb;
  display: flex;
  justify-content: center;
  align-items: center;
  font-weight: bold;
}

.user-box p {
  margin: 4px 0 0;
  font-size: 12px;
  color: #d1d5db;
}

.create-room {
  display: flex;
  gap: 8px;
}

.create-room input {
```

```
  flex: 1;
  padding: 10px;
  border-radius: 8px;
  border: none;
}

.room-list {
  display: flex;
  flex-direction: column;
  gap: 8px;
  overflow-y: auto;
}

.room {
  text-align: left;
}

.active-room {
  background: #2563eb !important;
}

.chat-area {
  flex: 1;
  display: flex;
  flex-direction: column;
  background: #f9fafb;
}

.empty-chat {
  height: 100%;
  display: flex;
  justify-content: center;
  align-items: center;
  color: #6b7280;
}

.chat-header {
  padding: 16px;
  background: white;
  border-bottom: 1px solid #e5e7eb;
  display: flex;
  justify-content: space-between;
  gap: 16px;
}

.chat-header h2 {
  margin: 0;
}

.chat-header p {
  margin: 4px 0 0;
  font-size: 12px;
  color: #6b7280;
}

.invite-box {
  display: flex;
  gap: 8px;
}

.invite-box input,
.search-box input,
.chat-input input {
  padding: 10px;
  border-radius: 10px;
  border: 1px solid #d1d5db;
}

.invite-box button,
.chat-input button,
```

```
.modal button {
  padding: 10px 14px;
  border-radius: 10px;
  border: none;
  background: #2563eb;
  color: white;
}

.search-box {
  padding: 10px 16px;
  background: white;
}

.search-box input {
  width: 100%;
}

.message-list {
  flex: 1;
  overflow-y: auto;
  padding: 18px;
}

.message-row {
  display: flex;
  margin-bottom: 12px;
}

.message-row.mine {
  justify-content: flex-end;
}

.message-bubble {
  max-width: 55%;
  background: white;
  padding: 12px;
  border-radius: 16px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.06);
  animation: popIn 0.2s ease;
}

.message-row.mine .message-bubble {
  background: #dbeafe;
}

.message-sender {
  display: flex;
  align-items: center;
  gap: 6px;
  font-size: 12px;
  color: #6b7280;
  margin-bottom: 6px;
}

.message-sender img {
  width: 22px;
  height: 22px;
  border-radius: 50%;
  object-fit: cover;
}

.message-bubble p {
  margin: 0;
  word-break: break-word;
}

.message-bubble small {
  display: block;
  margin-top: 4px;
  color: #6b7280;
```

```
}

.sent-image {
  max-width: 220px;
  max-height: 220px;
  border-radius: 10px;
}

.message-actions {
  margin-top: 8px;
  display: flex;
  gap: 8px;
}

.message-actions button {
  font-size: 12px;
  border: none;
  background: #e5e7eb;
  padding: 6px 8px;
  border-radius: 8px;
}

.chat-input {
  padding: 14px;
  background: white;
  border-top: 1px solid #e5e7eb;
  display: flex;
  gap: 8px;
}

.chat-input input[type="text"],
.chat-input input:not([type]) {
  flex: 1;
}

.modal-bg {
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.45);
  display: flex;
  justify-content: center;
  align-items: center;
}

.modal {
  width: 380px;
  background: white;
  border-radius: 18px;
  padding: 22px;
  display: flex;
  flex-direction: column;
  gap: 12px;
  animation: fadeIn 0.25s ease;
}

.modal input,
.modal textarea {
  padding: 10px;
  border: 1px solid #d1d5db;
  border-radius: 10px;
}

.modal textarea {
  min-height: 100px;
}

.profile-preview {
  width: 80px;
  height: 80px;
  border-radius: 50%;
```

```
  align-self: center;
  object-fit: cover;
}

.modal-buttons {
  display: flex;
  justify-content: flex-end;
  gap: 10px;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(8px);
  }

  to {
    opacity: 1;
    transform: translateY(0);
  }
}

@keyframes popIn {
  from {
    opacity: 0;
    transform: scale(0.96);
  }

  to {
    opacity: 1;
    transform: scale(1);
  }
}

@media (max-width: 768px) {
  .app {
    flex-direction: column;
  }

  .sidebar {
    width: 100%;
    height: 260px;
  }

  .chat-header {
    flex-direction: column;
  }

  .message-bubble {
    max-width: 85%;
  }

  .chat-input {
    flex-wrap: wrap;
  }

  .chat-input input {
    flex: 1;
  }
}
```

Firestore Rules

In Firebase Console → Firestore Database → Rules:

```
rules_version = '2';
```

```
service cloud.firestore {
  match /databases/{database}/documents {

    match /users/{userId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null && request.auth.uid == userId;
    }

    match /chatrooms/{roomId} {
      allow read, update, delete: if request.auth != null &&
        request.auth.uid in resource.data.members;

      allow create: if request.auth != null;

      match /messages/{messageId} {
        allow read: if request.auth != null;
        allow create: if request.auth != null;
        allow update, delete: if request.auth != null &&
          request.auth.uid == resource.data.senderId;
      }
    }
  }
}
```

Prompt: “okay now for the chat...can you make it bubbles? now it's just square right”
Answer:

Yes. Add this **near the bottom of** `App.css`, after the readable chat font code:

CSS

```css
.message-bubble {
  border-radius: 20px 20px 20px 6px !important;
}

.message-row.mine .message-bubble {
  border-radius: 20px 20px 6px 20px !important;
}

.sent-image {
  border-radius: 14px !important;
}

.message-sender img,
.user-box img,
.avatar,
.profile-preview {
  border-radius: 50% !important;
}
```

This keeps the pixel login page, but makes the chat messages look like real chat bubbles.

Prompt and Answer:

– Check your website working fine on different size device
– All components must remain visible — nothing should disappear
– If the layout requires scrolling on smaller screens, points will be deducted this part still bugs

Yeah, your current mobile layout may still lose points because the sidebar becomes tall and the page may need scrolling.

For this requirement, use a **fixed two-column layout even on small screens**, just narrower. Add this at the **very bottom of** `App.css` to override the old mobile rules:

CSS

```css
@media (max-width: 850px) {
  .app {
    flex-direction: row !important;
    height: 100vh;
    padding: 6px;
    gap: 6px;
    overflow: hidden;
  }

  .sidebar {
    width: 34vw !important;
    min-width: 145px;
    height: calc(100vh - 12px) !important;
    padding: 8px;
    gap: 7px;
    overflow: hidden;
  }

  .chat-area {
    width: 66vw;
    height: calc(100vh - 12px);
    min-width: 0;
  }

  .user-box {
    padding: 6px;
    gap: 6px;
```

Prompt and Answer:

is there anything that i need to delete

Yes. Delete or override the **old mobile media queries** near the bottom of `App.css`.

Look for these:

```css
@media (max-width: 850px) {
  ...
}

@media (max-width: 520px) {
  ...
}
```

You have older ones that make this happen:

```css
.app {
  flex-direction: column;
}
```

and:

```css
.sidebar {
  width: 100%;
  height: 270px;
}
```

Delete those old `@media` blocks, then paste the new RWD code at the bottom.

Also delete this if you see it inside old mobile CSS:

```css
.chat-input button {
  width: 100%;
}
```

Prompt and Answer:

yeah i can't use storage in firebase because i'm only using the free one. any other way

Thought for 4s >

Yes. Use **Base64**, but compress/resize the image before saving it to Firestore.

Your old method probably failed because the image was too large.

Replace your current `sendImage` function with this:

JavaScript

```
function resizeImage(file, maxWidth = 700, quality = 0.65) {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();

    reader.onload = event => {
      const img = new Image();

      img.onload = () => {
        const canvas = document.createElement("canvas");

        let width = img.width;
        let height = img.height;

        if (width > maxWidth) {
          height = (maxWidth / width) * height;
          width = maxWidth;
        }

        canvas.width = width;
        canvas.height = height;

        const ctx = canvas.getContext("2d");
        ctx.drawImage(img, 0, 0, width, height);

        const compressedBase64 = canvas.toDataURL("image/jpeg", quality);
        resolve(compressedBase64);
      };

      img.onerror = reject;
```

JavaScript

```
  if (!file.type.startsWith("image/")) {
    alert("Please choose an image file.");
    return;
  }

  try {
    const compressedImage = await resizeImage(file);

    await addDoc(collection(db, "chatrooms", currentRoom.id, "messages"), {
      image: compressedImage,
      senderId: user.uid,
      senderEmail: user.email,
      senderName: profile?.username || user.email,
      senderPhoto: profile?.profilePicture || "",
      type: "image",
      createdAt: serverTimestamp(),
      edited: false
    });

    e.target.value = "";
  } catch (error) {
    console.error(error);
    alert("Image upload failed. Please try a smaller image.");
  }
}
```

So keep this part in your message display:

JavaScript

```
{message.type === "image" ? (
  <img
    className="sent-image"
    src={message.image}
    alt="sent"
  />
) : (
  <p>{message.text}</p>
)}
```

This avoids Firebase Storage and saves a smaller image directly inside Firestore. Use smaller images if it still fails.

Prompt and Answer:

okay good. the unsend and edit button is always shown in the chat bubble right. i want it to be shown when i hover on the chat instead instead of always shown. is this possible

Yes. Add this to the **bottom of** `App.css`:

CSS

```css
.message-actions {
  opacity: 0;
  visibility: hidden;
  transform: translateY(4px);
  transition: opacity 0.18s ease, transform 0.18s ease, visibility 0.18s ease;
}

.message-row.mine:hover .message-actions {
  opacity: 1;
  visibility: visible;
  transform: translateY(0);
}
```

This keeps **Edit / Unsend** hidden normally and only shows them when you hover over your own message bubble.

Prompt and Answer:

okay i already clicked on firebase so people can log in using google.. but right now there's no button for it

Thought for 10s

Yes. Add the button and function.

In `App.js`, update your auth import:

JavaScript

```javascript
import {
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signOut,
  onAuthStateChanged,
  GoogleAuthProvider,
  signInWithPopup
} from "firebase/auth";
```

Then add this function inside `App()`:

JavaScript

```
async function handleGoogleLogin() {
  const provider = new GoogleAuthProvider();
  const result = await signInWithPopup(auth, provider);
  const googleUser = result.user;

  const userRef = doc(db, "users", googleUser.uid);
  const userSnap = await getDoc(userRef);

  if (!userSnap.exists()) {
    const newProfile = {
      uid: googleUser.uid,
      email: googleUser.email,
      username: googleUser.displayName || googleUser.email.split("@")[0],
      phone: "",
      address: "",
      profilePicture: googleUser.photoURL || ""
    };

    await setDoc(userRef, newProfile);
    setProfile(newProfile);
  } else {
    setProfile(userSnap.data());
  }
}
```

Firebase's official web method for Google login is `GoogleAuthProvider` with `signInWithPopup` Firebase +1.

Then in your login/signup form, put this **under the email/password submit button**:

JavaScript

```
<button type="button" onClick={handleGoogleLogin}>
  Continue with Google
</button>
```

So this part becomes:

JavaScript

```
<button type="submit">
  {mode === "login" ? "Sign In" : "Create Account"}
</button>

<button type="button" onClick={handleGoogleLogin}>
  Continue with Google
</button>
```

That's it. Google login should now appear on the sign in/sign up page.

Prompt: “okay..help me with the notifications part please.”
Answer:

We can add this without Firebase Cloud Messaging. For the assignment’s Chrome notification requirement, a browser `Notification` API solution is enough for local chat notifications, and we’ll make it trigger only for other people’s new/unread messages.

Thought for 5s ›

Yes. Add this to your current `App.js`.

1. Add these refs near `bottomRef`

JavaScript

```
const notifiedMessagesRef = useRef(new Set());
const firstLoadRoomsRef = useRef(new Set());
const currentRoomRef = useRef(null);
```

2. Add this `useEffect`

JavaScript

```
useEffect(() => {
  currentRoomRef.current = currentRoom;
}, [currentRoom]);
```

3. Add this function inside `App()`

JavaScript

```
function requestNotificationPermission() {
  if (!("Notification" in window)) {
    alert("This browser does not support notifications.");
    return;
  }

  if (Notification.permission === "default") {
    Notification.requestPermission();
  }
}
```

4. Add this `useEffect` after your chatroom loading `useEffect`

JavaScript

```
useEffect(() => {
  if (!user) return;

  const unsubscribers = [];

  chatrooms.forEach(room => {
    const q = query(
      collection(db, "chatrooms", room.id, "messages"),
      orderBy("createdAt", "asc")
    );

    const unsubscribe = onSnapshot(q, snapshot => {
      if (!firstLoadRoomsRef.current.has(room.id)) {
        snapshot.docs.forEach(docSnap => {
          notifiedMessagesRef.current.add(docSnap.id);
        });

        firstLoadRoomsRef.current.add(room.id);
        return;
      }

      snapshot.docChanges().forEach(change => {
        if (change.type !== "added") return;

        const message = change.doc.data();
        const messageId = change.doc.id;

        if (notifiedMessagesRef.current.has(messageId)) return;

        notifiedMessagesRef.current.add(messageId);

        const isMine = message.senderId === user.uid;
        const isCurrentRoom = currentRoomRef.current?.id === room.id;

        if (!isMine && !isCurrentRoom && Notification.permission === "granted") {
          new Notification(`New message in ${room.name}`, {
            body:
              message.type === "image"
                ? `${message.senderName || message.senderEmail} sent an image`
                : `${message.senderName || message.senderEmail}: ${message.text}`,
          });
```

JavaScript

```
      });
    });

    unsubscribers.push(unsubscribe);
  });

  return () => {
    unsubscribers.forEach(unsubscribe => unsubscribe());
  };
}, [user, chatrooms]);
```

5. Add a notification button in the sidebar

Put this under your `Logout` button:

JavaScript

```
<button onClick={requestNotificationPermission}>
  Enable Notifications
</button>
```

So this part becomes:

JavaScript

```
<button onClick={() => setShowProfile(true)}>Edit Profile</button>
<button onClick={handleLogout}>Logout</button>
<button onClick={requestNotificationPermission}>
  Enable Notifications
</button>
```

Prompt and Answer:

Chatbot (2%) (Must use ChatGPT or gemini api) help please

We can add the chatbot as a special "Bot" chatroom member that replies when the user types a message starting with `@bot`. I'll use Gemini because it is easier to connect from a frontend React project for this assignment.

Thought for 8s >

Yes. Easiest: use **Gemini API** and make the bot reply when you type:

```
@bot your question here
```

Gemini's current JavaScript SDK is `@google/genai`, and Google warns not to expose API keys in production frontend code, but for a class demo this is usually acceptable if documented. GitHub

1. Install Gemini SDK

```bash
npm install @google/genai
```

2. Create `.env` in your project root

Same level as `package.json`:

```env
REACT_APP_GEMINI_API_KEY=your_gemini_api_key_here
```

Then restart:

```bash
npm start
```

3. Add import in `App.js`

At the top:

```javascript
import { GoogleGenAI } from "@google/genai";
```

4. Add this function inside `App()`

Put it near `sendMessage()`:

JavaScript

```
async function getBotReply(userText) {
  try {
    const ai = new GoogleGenAI({
      apiKey: process.env.REACT_APP_GEMINI_API_KEY
    });

    const response = await ai.models.generateContent({
      model: "gemini-2.5-flash",
      contents: userText
    });

    return response.text || "Sorry, I could not answer that.";
  } catch (error) {
    console.error(error);
    return "Sorry, the chatbot is not available right now.";
  }
}
```

5. Replace your `sendMessage()` with this

JavaScript

```
async function sendMessage() {
  const textToSend = messageText.trim();

  if (!currentRoom || textToSend === "") return;

  setMessageText("");

  await addDoc(collection(db, "chatrooms", currentRoom.id, "messages"), {
    text: textToSend,
    senderId: user.uid,
    senderEmail: user.email,
    senderName: profile?.username || user.email,
    senderPhoto: profile?.profilePicture || "",
    type: "text",
    createdAt: serverTimestamp(),
    edited: false
```

JavaScript

```
  });

  if (textToSend.toLowerCase().startsWith("@bot")) {
    const prompt = textToSend.replace(/^@bot/i, "").trim();

    if (prompt === "") return;

    await addDoc(collection(db, "chatrooms", currentRoom.id, "messages"), {
      text: "Typing...",
      senderId: "chatbot",
      senderEmail: "chatbot@gemini.ai",
      senderName: "Gemini Bot",
      senderPhoto: "",
      type: "text",
      createdAt: serverTimestamp(),
      edited: false
    });

    const botReply = await getBotReply(prompt);

    await addDoc(collection(db, "chatrooms", currentRoom.id, "messages"), {
      text: botReply,
      senderId: "chatbot",
      senderEmail: "chatbot@gemini.ai",
      senderName: "Gemini Bot",
      senderPhoto: "",
      type: "text",
      createdAt: serverTimestamp(),
      edited: false
    });
  }
}
```

Prompt and Answer:

---

is there any way to make it reply more than once

Yes. Two options:

Option 1: reply every time you type `@bot`
This is already how it works:

```
@bot hello
@bot explain this
@bot give me an example
```

Bot replies each time.

Option 2: make a dedicated Bot chatroom where every message gets a bot reply
Change this line in `sendMessage()`:

```javascript
if (textToSend.toLowerCase().startsWith("@bot")) {
```

to:

```javascript
if (
  textToSend.toLowerCase().startsWith("@bot") ||
  currentRoom.name.toLowerCase().includes("bot")
) {
```

Then create a room called:

```
Bot Room
```

In that room, the bot will reply to every message, even without `@bot`.

Prompt and Answer:

Use CSS animation
– https://www.youtube.com/watch?v=bjUoQbSJDJs
– Button hover is not an animation! what about this one

Yeah this one is stricter than it looks. They want **real CSS animation**, not just `:hover`.

You need something like:

- typing dots animation
- loading animation
- fade/slide animation
- pulse effect

Easiest win (use your bot typing 👇)

Turn this:

```
Gemini Bot is typing...
```

into an animated typing dots effect

---

1. Replace your typing UI

Change this:

JavaScript

```javascript
{botTyping && (
  <div className="bot-typing-row">
    Gemini Bot is typing...
  </div>
)}
```

to:

JavaScript

```javascript
{botTyping && (
  <div className="bot-typing-row">
    <span className="typing-text">Gemini Bot is typing</span>
    <span className="dots">
      <span>.</span>
      <span>.</span>
      <span>.</span>
    </span>
  </div>
)}
```

2. Add this to `App.css`

CSS

```css
.bot-typing-row {
  margin-left: 52px;
  margin-bottom: 14px;
  color: #6b7280;
  font-size: 14px;
  display: flex;
  align-items: center;
  gap: 6px;
}

/* dots container */
.dots span {
  opacity: 0;
  animation: blink 1.4s infinite;
  font-weight: bold;
}

/* delay each dot */
.dots span:nth-child(1) {
  animation-delay: 0s;
}
.dots span:nth-child(2) {
  animation-delay: 0.2s;
}
.dots span:nth-child(3) {
  animation-delay: 0.4s;
}

/* keyframes animation */
@keyframes blink {
  0% { opacity: 0; }
  20% { opacity: 1; }
  100% { opacity: 0; }
}
```

Prompt and Answer:

okay now help me with this first please: Reply for specify message(6%)
• User can reply for specify message (include the message send by himself/herself) and be sure have the clear UI to show the replied message and original message. (3%)
• Need to show the replied message above the text input when user is typing the response (1%)
• Clicking a replied message should trigger a visual highlight on the original message to indicate focus.(1%)
• Clicking a replied message should scroll the chat to the original message if it's out of view.

We can add this without touching Firebase rules much: each message will store a small `replyTo` object, and the UI will keep refs to message bubbles so clicking a reply can scroll/highlight the original.

Thought for 5s ›

Yes. Implement it like this.

What to add

Each replied message should save:

JavaScript

```
replyTo: {
  id: originalMessage.id,
  text: originalMessage.text || "Image",
  senderName: originalMessage.senderName
}
```

You need 4 parts:

```
1. reply state
2. reply button on each message
3. reply preview above input
4. scroll + highlight original message
```

Main logic to add in `App.js`

Add states/refs:

JavaScript

```
const [replyTo, setReplyTo] = useState(null);
const [highlightedMessageId, setHighlightedMessageId] = useState(null);
const messageRefs = useRef({});
```

Add functions:

JavaScript

```
function startReply(message) {
  setReplyTo({
    id: message.id,
    text: message.type === "image" ? "Image" : message.text,
    senderName: message.senderName || message.senderEmail
  });
}

function focusOriginalMessage(messageId) {
  const target = messageRefs.current[messageId];

  if (target) {
    target.scrollIntoView({
      behavior: "smooth",
      block: "center"
    });

    setHighlightedMessageId(messageId);

    setTimeout(() => {
      setHighlightedMessageId(null);
    }, 1200);
  }
}
```

In `sendMessage()`, add `replyTo`:

JavaScript

```
await addDoc(collection(db, "chatrooms", currentRoom.id, "messages"), {
  text: textToSend,
  senderId: user.uid,
  senderEmail: user.email,
  senderName: profile?.username || user.email,
  senderPhoto: profile?.profilePicture || "",
  type: "text",
  replyTo: replyTo,
  createdAt: serverTimestamp(),
  edited: false
});
```

Inside each message row, add ref + highlight class:

JavaScript

```
<div
  key={message.id}
  ref={el => {
    messageRefs.current[message.id] = el;
  }}
  className={
    `${isMine ? "message-row mine" : "message-row"} ${
      highlightedMessageId === message.id ? "highlight-message" : ""
    }`
  }
>
```

Inside `.message-bubble`, before the actual message text/image, add:

JavaScript

```
{message.replyTo && (
  <div
    className="reply-preview-in-message"
    onClick={() => focusOriginalMessage(message.replyTo.id)}
  >
    <strong>{message.replyTo.senderName}</strong>
    <span>{message.replyTo.text}</span>
  </div>
)}
```

In your message actions, add:

JavaScript

```
<button onClick={() => startReply(message)}>
  Reply
</button>
```

For non-mine messages, also add a reply button under the bubble:

JavaScript

```
{!isMine && (
  <div className="message-actions show-reply">
    <button onClick={() => startReply(message)}>
      Reply
    </button>
  </div>
)}
```

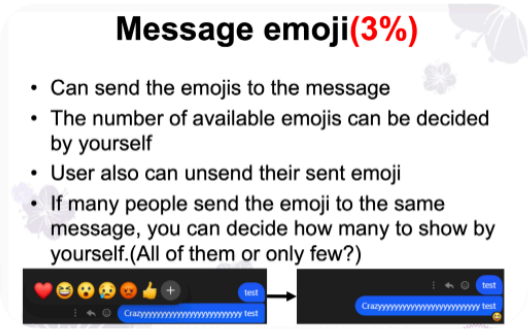
Above your chat input, add:

JavaScript

```
{replyTo && (
  <div className="reply-preview-input">
    <div>
      <strong>Replying to {replyTo.senderName}</strong>
      <p>{replyTo.text}</p>
    </div>

    <button type="button" onClick={() => setReplyTo(null)}>
      X
    </button>
  </div>
)}
```

Prompt and Answer:



need help

Thought for 7s ›

Yes. This means **emoji reactions on each message**, like Messenger/LINE.

Simplest version that satisfies it:

```
1. Each message has emoji buttons
2. User can react to a message
3. User can click same emoji again to unsend/remove their emoji
4. Show emoji count under the message
```

Add to each message data

Each message can have:

JavaScript

```
reactions: {
  "👍": ["uid1", "uid2"],
  "😂": ["uid3"]
}
```

Add this function in `App.js`

JavaScript

```
async function toggleReaction(message, emoji) {
  if (!currentRoom || !user) return;

  const messageRef = doc(
    db,
    "chatrooms",
    currentRoom.id,
    "messages",
    message.id
  );

  const oldReactions = message.reactions || {};
  const oldUsers = oldReactions[emoji] || [];

  let updatedUsers;

  if (oldUsers.includes(user.uid)) {
    updatedUsers = oldUsers.filter(uid => uid !== user.uid);
  } else {
    updatedUsers = [...oldUsers, user.uid];
  }
```

JavaScript

```
  const updatedReactions = {
    ...oldReactions,
    [emoji]: updatedUsers
  };

  if (updatedUsers.length === 0) {
    delete updatedReactions[emoji];
  }

  await updateDoc(messageRef, {
    reactions: updatedReactions
  });
}
```

Add this inside each `.message-bubble`

Put it **after the message text/image**, before edited/actions:

JavaScript

```
<div className="emoji-bar">
  {["❤️", "😂", "😮", "😭", "😡", "👍"].map(emoji => (
    <button
      key={emoji}
      type="button"
      onClick={() => toggleReaction(message, emoji)}
    >
      {emoji}
    </button>
  ))}
</div>

{message.reactions && (
  <div className="reaction-display">
    {Object.entries(message.reactions).map(([emoji, users]) => (
      users.length > 0 && (
        <span key={emoji}>
          {emoji} {users.length}
        </span>
      )
    ))}
  </div>
```

CSS

```css
.emoji-bar {
  display: none;
  gap: 5px;
  margin-top: 8px;
}

.message-bubble:hover .emoji-bar {
  display: flex;
}

.emoji-bar button {
  border: none;
  box-shadow: none;
  background: #f3f4f6;
  padding: 4px 6px;
  border-radius: 999px;
  font-size: 15px;
}

.reaction-display {
  display: flex;
  gap: 6px;
  margin-top: 7px;
  flex-wrap: wrap;
}

.reaction-display span {
  background: #ffffff;
  color: #111827;
  border: 2px solid #111827;
  border-radius: 999px;
  padding: 2px 7px;
  font-size: 13px;
}
```

This should satisfy:

```
Can send emoji to message
You decide available emojis
User can unsend their emoji by clicking same emoji again
Many people can react, shown as count
```

Prompt and Answer:

this happen when i press an emoji

Thought for 5s

This is from **Firestore Rules**, not your JS.

Right now your rules probably only allow the message sender to update/delete their own message. But emoji reaction needs to update **any message**, even if it was sent by someone else.

Go to:

```
Firebase Console → Firestore Database → Rules
```

Replace your `messages` rule with this:

JavaScript

```
match /chatrooms/{roomId}/messages/{messageId} {
  allow read: if request.auth != null;

  allow create: if request.auth != null;

  allow update: if request.auth != null &&
    (
      request.auth.uid == resource.data.senderId ||
      request.resource.data.diff(resource.data).affectedKeys().hasOnly(["reactions"])
    );

  allow delete: if request.auth != null &&
    request.auth.uid == resource.data.senderId;
}
```

Then click **Publish**.

Prompt: “i still can send more than one emoji per message. i want it to be when i already send a heart. i can't send another emoji. unless i unclick the heart one.”
Answer:

Thought for 8s

Replace only your `toggleReaction()` function in `App.js`. Your current JS already has the Facebook-style reaction button outside the bubble

JavaScript

```javascript
async function toggleReaction(message, emoji) {
  if (!currentRoom || !user) return;

  const messageRef = doc(
    db,
    "chatrooms",
    currentRoom.id,
    "messages",
    message.id
  );

  const oldReactions = message.reactions || {};

  let userCurrentEmoji = null;

  Object.entries(oldReactions).forEach(([reactionEmoji, users]) => {
    if (users.includes(user.uid)) {
      userCurrentEmoji = reactionEmoji;
    }
  });

  if (userCurrentEmoji && userCurrentEmoji !== emoji) {
    alert("You already reacted to this message. Remove your current emoji first.");
    return;
  }

  const updatedReactions = {};

  Object.entries(oldReactions).forEach(([reactionEmoji, users]) => {
    let updatedUsers = users;

    if (reactionEmoji === emoji && users.includes(user.uid)) {
      updatedUsers = users.filter(uid => uid !== user.uid);
    } else if (reactionEmoji === emoji && !users.includes(user.uid)) {
      updatedUsers = [...users, user.uid];
    }

    if (updatedUsers.length > 0) {
      updatedReactions[reactionEmoji] = updatedUsers;
    }
  });
```

JavaScript

```
    }
  });

  if (!oldReactions[emoji] && !userCurrentEmoji) {
    updatedReactions[emoji] = [user.uid];
  }

  await updateDoc(messageRef, {
    reactions: updatedReactions
  });
}
```

Now it works like:

```
Heart selected → cannot select 😂
Click heart again → heart removed
Then you can select another emoji
```